eMerger

One command to update your whole system

Roberto Antoniello & Edoardo Ferrari

April 14, 2026

Abstract

eMerger is a cross-platform command line tool that refreshes, upgrades and cleans a computer with a single command: `up`. It runs on Linux, macOS and Windows, auto-detects every package manager present on the host and invokes them in the right order, returning one unified summary at the end. This document describes the architecture, features, configuration and integration points of the current release.

Contents

---

1 Overview

eMerger began as a small Bash experiment and has grown into a portable updater with a shared feature set across the three major desktop and server platforms. The design follows three principles:

- **One command.** The user types `up`; everything else is automatic detection, ordering and reporting.
- **No runtime bloat.** No daemon, no YAML, no embedded DSL. Configuration is a plain shell or PowerShell file that is sourced at startup.
- **Use what is there.** eMerger never installs a package manager on your behalf; it detects the ones already installed and delegates work to them.

The project name is a wink at Gentoo's `emerge`: the tool started under the name *Updater* and was later renamed to *eMerger* to reflect its goal of merging the quirks of every supported package manager behind a uniform interface.

2 Platforms

	Linux	macOS	Windows
Entry point	`src/emerger.sh`	`src/emerger.sh`	`src/emerger.ps1`
Shell	bash 3.2+	bash 3.2+ (system)	PowerShell 5.1+
Setup	`./setup.sh`	`./setup.sh`	`.\setup.ps1`
Uninstall	`./uninstall.sh`	`./uninstall.sh`	`.\uninstall.ps1`
Auto-update	systemd timer / cron	cron	Task Scheduler
Elevation	sudo	sudo	UAC relaunch
Config dir	`~/.config/emerger/`	`~/.config/emerger/`	`%APPDATA%\emerger\`
State dir	`~/.local/state/emerger/`	`~/.local/state/emerger/`	`%LOCALAPPDATA%\emerger\state\`

The core flow (detect, upgrade, clean, summarise) is kept at parity across platforms. Features that depend on operating system primitives, such as firmware updates or filesystem snapshots, are gated to the platforms that support them and are marked as such throughout this document.

3 Installation

3.1 Linux and macOS

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
./setup.sh
# open a new shell (or: source ~/.bashrc)
up --help
```

`setup.sh` performs the following actions:

1. Adds `alias up='bash /path/to/eMerger/src/emerger.sh'` to the appropriate shell rc file (bash, zsh or fish).
2. Marks `src/emerger.sh` as executable.
3. Installs shell completions for bash, zsh and fish.
4. Scaffolds `~/.config/emerger/` with the default `config.sh`, `ignore.list`, `hooks/` and `profiles.d/` layout.

On macOS the alias is written to `~/.zshrc` first, since zsh is the default interactive shell from Catalina onwards. When Homebrew is present, completions are installed into the Homebrew prefix so they are loaded by the brew-managed shell environment.

3.2 Windows

```
git clone https://github.com/MasterCruelty/eMerger
cd eMerger
.\setup.ps1
# open a new PowerShell window (or: . $PROFILE)
up --help
```

`setup.ps1` does not require administrator privileges. It lifts the execution policy for the current user to `RemoteSigned` when needed, defines a `function up` inside `$PROFILE.CurrentUserAllHosts` so that the command is available from every PowerShell host, and scaffolds `%APPDATA%\emerger\`. When a package manager invocation later needs elevation, eMerger relaunches itself under UAC.

3.3 Manual install

When a packaged setup is undesirable, pointing an alias at the entry point is enough:

```
alias up='bash /absolute/path/to/eMerger/src/emerger.sh'
```

```
function up { & "C:\path\to\eMerger\src\emerger.ps1" @args }
```

3.4 Uninstall

Running `./uninstall.sh` (Unix) or `.\uninstall.ps1` (Windows) removes the shell alias, the cron job or systemd timer, and the scheduled task. Configuration and state directories are preserved so that logs, profiles and hooks survive a reinstall; remove them manually to start from a clean slate.

4 Self-update

Three equivalent forms are accepted:

```
up -up
up --self-update
./update.sh        # .\update.ps1 on Windows
```

Internally a `git pull -ff-only` is performed inside the clone and the new commit range is printed. Non fast-forward updates are refused to guarantee that local changes cannot silently disappear.

5 Execution flow

A full run of `up` performs, in order:

1. Load `config.sh` (or `config.ps1`) and any `--profile`.
2. Parse CLI flags; CLI always wins over config and profile defaults.
3. Acquire an exclusive lock (`/tmp/emerger.lock` on Unix).
4. Print the logo, OS info line and timestamp.
5. Warn on low battery and on low free disk space.
6. Cache `sudo` credentials on Unix, or relaunch elevated on Windows, only when a detected manager actually needs it.
7. Snapshot the list of installed packages for the post-run diff.
8. Execute `pre.d` hooks.
9. For each detected manager, run *refresh*, *upgrade*, *clean*.
10. Optionally clear user cache and trash.
11. Execute `post.d` hooks.
12. Compute the installed-packages diff.
13. Render the summary, the reboot advisory and an optional desktop notification.

6 Command line reference

The authoritative reference is `up --help`. The most frequently used flags are summarised below.

Flag	Meaning	Platforms
`-n`, `--dry-run`	Preview without running	all
`-v`, `--verbose`	Stream manager output live	all
`-q` / `-qq` / `-qqq`	Progressive quiet levels	all
`-y`, `--yes`	Assume yes on prompts	all
`-i`, `--interactive`	TUI menu (gum / whiptail)	Unix
`--security`	Security-only updates	supported managers
`--dev`	Include dev toolchains	all
`--firmware`	`fwupdmgr` refresh+upd	Linux
`--parallel`	Run user-space in parallel	Unix
`--profile NAME`	Load profile	all
`--snapshot`	snapper / timeshift / btrfs	Linux
`--refresh-mirrors`	Re-rank distribution mirrors	Linux
`--resume`	Skip completed managers	Unix
`--reboot`	Reboot if required	all
`--changed`	Show package diff	Unix
`--changelog PKG`	Upstream changelog for PKG	Unix
`--report FILE`	Export Markdown report	Unix
`--doctor`	Health check	all
`--history`	Recent runs	all
`--errors`, `-err`	Tail of logged errors	all
`--json`	Machine-readable summary	all
`--reboot-exit`	Exit 4 if reboot is required	all
`--rollback`	Revert last snapshot	Linux
`--download-only`, `--offline`	Prefetch, don't install	Unix
`--only LIST`	Only these managers	all
`--except LIST`	Skip these managers	all
`--metrics FILE`	Prometheus textfile export	all
`-up`	Self-update eMerger	all
`-au`	Install weekly auto-update	all

6.1 Combining flags

Flags are independent tokens and can be combined freely:

```
up -n -v                    # preview a full run, live stream
up -y -q --security         # unattended security-only, minimal output
up --dev --parallel -v      # include dev toolchains, parallel user-space
up --snapshot --reboot -y   # snapshot first, reboot at the end if needed
up --profile server --resume
```

Single-letter short flags in the set `{h V n v q y i w}` also bundle: `up -nv` equals `up -n -v`. Compound short flags (`-nl`, `-ni`, `-qq`, `-up`, `-err`, ...) and long flags are never bundled.

7 Features in depth

7.1 Dry-run and verbose

`-n` prints every command that would be executed without actually running it or asking for elevation, which makes it safe to run in a terminal that does not own sudo credentials. `-v` streams the output of each package manager live instead of buffering it into the summary. The two combine naturally.

7.2 Quiet levels

`-q` hides informational output, `-qq` keeps only the step titles and the final one-line summary, `-qqq` produces no output at all and communicates only through the exit code.

7.3 Security-only updates

When `--security` is passed, each supporting manager is asked to limit the upgrade set to security advisories: `apt` via `unattended-upgrade`, `dnf` with `--security`, `zypper patch --category security`, macOS via `softwareupdate --recommended` and Windows via `PSWindowsUpdate` severity filtering. Managers that do not distinguish between security and non-security updates ignore the flag.

7.4 Dev toolchains

`--dev` enables a separate track that runs `rustup` updates, `cargo install-update -a`, `npm update -g`, `pnpm -g update`, `pip` user site upgrades and `gem update`. These run under the invoking user, never under sudo.

7.5 Firmware (Linux)

`--firmware` runs `fwupdmgr refresh && fwupdmgr update -y --no-reboot-check`. On macOS firmware is handled by `softwareupdate`; on Windows it is delegated to vendor tooling (Dell Command Update, Lenovo Vantage, etc.) and is out of scope.

7.6 Parallel mode

`--parallel` runs user-space managers that do not touch the root filesystem concurrently: `flatpak`, `snap`, `brew`, `mas` and dev toolchains. System managers stay serial to avoid lock contention. The Windows side is currently always serial.

7.7 Snapshots and rollback (Linux)

`--snapshot` tries, in order, `snapper`, `timeshift` and raw `btrfs subvolume snapshot`. `--rollback` reverts to the most recent eMerger-created snapshot; `timeshift` is always invoked interactively because a silent rollback is too destructive to automate.

7.8 Mirrors refresh (Linux)

`--refresh-mirrors` re-ranks the repository mirrors before the upgrade: `reflector --latest 20 --sort rate` on Arch, `netselect-apt` on Debian and Ubuntu, and a no-op on Fedora where `fastestmirror` already handles ranking at the DNF level.

7.9 Reboot handling

On Unix, eMerger inspects `/var/run/reboot-required` and `needs-restarting -r`. On Windows it reads the CBS `RebootPending`, `WindowsUpdate\RebootRequired` and `PendingFileRenameOperations`

registry entries. The summary prints a reboot advisory; `--reboot` actually reboots and `--reboot-exit` exits with code 4 so that an orchestrator can decide.

7.10 Resume

When a run is interrupted, a cursor is written to `~/.local/state/emerger/resume`. A subsequent `up --resume` skips every manager that completed successfully, so repeated failures do not force a full retry. Unix only.

7.11 Package diff and changelog

Each run records the installed package set before and after. `up --changed` prints the diff with `+`, `-` and `~` markers for added, removed and upgraded packages. `up --changelog PKG` dispatches to `apt changelog`, `dnf changelog`, `pacman -Qi` or `brew log` depending on the host.

7.12 Reports, history and errors

`--report FILE` exports a Markdown report of the last run. `--history` prints a table of the most recent runs. `--errors` tails `ERROR` lines from the log.

7.13 Doctor

`--doctor` audits the environment: shell or PowerShell version, sudo or admin state, disk space, network reachability, state directory writability, per-manager native health checks (`dpkg --audit`, `pacman -Dk`, `brew doctor`), pending reboot flags and, on Windows, the current execution policy. A non-zero exit code is returned when any issue is detected.

8 Configuration

8.1 Config file

Unix (`~/.config/emerger/config.sh`, sourced before argument parsing):

```
ARG_DEV=1                   # always include dev toolchains
ARG_WEATHER=1               # show weather widget
DISK_MIN_FREE_MB=2048       # require at least 2 GB free on /
QUIET_HOURS="23:00-07:00"   # skip scheduled runs in this window
RETRY_MAX=3                 # transient-error retry count
```

Windows (`%APPDATA%\emerger\config.ps1`, dot-sourced):

```
$script:ArgsGlobal.Dev      = $true
$script:ArgsGlobal.Security = $true
$script:ArgsGlobal.NoTrash  = $true
```

8.2 Profiles

Profiles are named configuration snippets loaded through `--profile NAME`. The shipped set lives under `share/profiles/` and includes `work`, `home`, `server` and `safe`. User overrides live in `~/.config/emerger/profiles` on Unix and `%APPDATA%\emerger\profiles.d\` on Windows, and always shadow the shipped defaults.

8.3 Hooks

Executable scripts in `hooks/pre.d/` and `hooks/post.d/` run before and after the update flow, in alphabetical order. A failing hook emits a warning but does not abort the run, so hooks are safe by default. Unix hooks are `*.sh` under bash; Windows hooks are `*.ps1` dot-sourced under PowerShell.

```
# ~/.config/emerger/hooks/pre.d/10-backup-dotfiles.sh
#!/usr/bin/env bash
rsync -a ~/.config/ ~/backups/dotfiles/
```

8.4 Ignore list (Linux)

`~/.config/emerger/ignore.list` accepts one package per line, with # comments. `pacman` honours it natively via `--ignore=`. For `apt`, `dnf` and `zypper` the file is advisory: the user must still hold the packages through the manager's native command (`apt-mark hold`, `dnf versionlock add`, `zypper al`).

8.5 Quiet hours

`QUIET_HOURS="HH:MM-HH:MM"` in `config.sh` makes scheduled runs that start inside the window exit immediately when `-y` is set, i.e. the ones coming from the timer. Interactive invocations are never blocked.

8.6 Manager plugins

Dropping a bash script in `~/.config/emerger/managers.d/<name>.sh` registers a custom manager without touching the repository. A minimal plugin declares `pm_<slug>_detect` and `pm_<slug>_run`; optional callbacks expose whether the manager needs sudo, can run in parallel, is gated by `--dev` and which icon to print. Plugins honour `--only`, `--except`, `--parallel`, `--dev`, the detection cache, hooks and the summary. A copy-pasteable template is provided in `share/plugins/example.sh`. Linux and macOS only.

9 Integration

9.1 JSON output

`up --json` emits a single-line JSON object on stdout, with the logo, info line and summary box suppressed. The same schema is what `history.jsonl` stores one line at a time:

```
{"ts":"2026-04-14T07:24:31Z","duration":42,"freed_kb":15360,
 "errors":0,"reboot":0,
 "managers":[{"name":"apt","result":"ok"},{"name":"flatpak","result":"ok"}]}
```

9.2 Prometheus metrics

`up --metrics FILE` reads the most recent entry of `history.jsonl` and writes a Prometheus textfile-collector snapshot. Exposed gauges include `emerger_last_run_timestamp_seconds`, `emerger_last_run_duratio`
`emerger_last_run_freed_bytes`, `emerger_last_run_errors`, `emerger_reboot_required` and one `emerger_manager_ok{manager="..."}` per manager of the last run. The flag does not trigger an update run; invoke it from a `post.d` hook or from the scheduler after `up` completes.

9.3 Download-only and offline

`--download-only` (alias `--offline`) refreshes the package indexes and downloads the pending upgrade set but does not install it. Supported natively on `pacman` (`-Syuw`), `apt` (`--download-only`), `dnf` (`--downloadonly`) and `zypper` (`update --download-only`). This is the safe prefetch option for metered connections and for pre-staging maintenance windows.

9.4 Manager filtering

`--only LIST` restricts the run to the listed comma-separated managers; `--except LIST` excludes them. Filtering is applied after detection and after `--dev` / `--firmware` gating, and `--except` always wins over `--only` when both mention the same name.

10 Auto-update (unattended)

`up -au` installs a weekly unattended run. On Linux the preferred backend is a systemd user timer (`~/.config/systemd/user/emerger.{service,timer}`); cron is used as fallback. On macOS cron is the default and `launchd` can be used manually. On Windows a `Register-ScheduledTask -TaskName eMerger` is created. The trigger fires weekly on Sunday at 10:00 with a ± one-hour randomised delay, and always uses `-y -q -nl -ni`. Pair with `QUIET_HOURS` to keep the machine quiet at night.

11 Files and paths

Linux and macOS

Path	Purpose
`~/.config/emerger/config.sh`	User defaults
`~/.config/emerger/profiles.d/`	User profiles
`~/.config/emerger/hooks/{pre,post}.d/`	Hook scripts
`~/.config/emerger/ignore.list`	Ignore list (pacman native)
`~/.config/emerger/managers.d/*.sh`	User-defined manager plugins
`~/.cache/emerger/detected`	Detection cache
`~/.local/state/emerger/emerger.log`	Log (rotated at 2000 lines)
`~/.local/state/emerger/history.jsonl`	One JSON record per run
`~/.local/state/emerger/resume`	Resume cursor
`/tmp/emerger.lock`	Global lock (`flock`)

Windows

Path	Purpose
`%APPDATA%\emerger\config.ps1`	User defaults
`%APPDATA%\emerger\profiles.d\`	User profiles
`%APPDATA%\emerger\hooks\{pre,post}.d\`	Hook scripts
`%LOCALAPPDATA%\emerger\cache\`	Detection cache
`%LOCALAPPDATA%\emerger\state\emerger.log`	Log
`%LOCALAPPDATA%\emerger\state\history.jsonl`	Run history

12 Source tree

The Bash and PowerShell implementations share the same layering: argument parsing, UI, logging, locking and per-manager dispatch are each isolated in their own file.

Bash (src/lib/) `args.sh`, `changelog.sh`, `clean.sh`, `diff.sh`, `disk.sh`, `doctor.sh`, `estimate.sh`, `hooks.sh`, `ignore.sh`, `lock.sh`, `log.sh`, `metrics.sh`, `mirrors.sh`, `notify.sh`, `packages.sh`, `plugins.sh`, `profiles.sh`, `progress.sh`, `reboot.sh`, `report.sh`, `resume.sh`, `retry.sh`, `run.sh`, `snapshot.sh`, `summary.sh`, `sys.sh`, `tui.sh`, `ui.sh`, `update.sh`, `wizard.sh`, `help.txt`.

PowerShell (src/pslib/) `Args.ps1`, `Clean.ps1`, `Doctor.ps1`, `Hooks.ps1`, `Log.ps1`, `Notify.ps1`, `Packages.ps1`, `Profiles.ps1`, `Summary.ps1`, `Sys.ps1`, `UI.ps1`, `Update.ps1`, `Help.txt`.

Adding a new package manager means extending `src/lib/packages.sh` on the Unix side or `src/pslib/Packages.ps1` on the Windows side; both are table-driven dispatchers.

13 Supported package managers

- **Linux, system (sudo):** `pacman`, `apt`, `apt-get`, `dnf`, `yum`, `zypper`, `xbps`, `apk`, `eopkg`, `emerge`, `nixos-rebuild`, `fwupdmgr`, `snap`.
- **Linux, AUR (no sudo):** `yay`, `paru`.
- **Linux, user-space:** `flatpak`, `nix-env`.
- **macOS:** `softwareupdate`, `brew`, `brew -cask`, `mas`.
- **Windows:** `winget`, `scoop`, `choco`, `PSWindowsUpdate`, `wsl -update`.
- **Dev toolchains (all platforms, opt-in):** `rustup`, `cargo install-update`, `npm`, `pnpm`, `pip`, `gem`.

14 Exit codes

Code	Meaning
0	Success
1	Runtime failure (sudo, lock, disk, interrupted)
2	Argument parsing error
3	One or more package managers returned non-zero
4	Reboot required (only with `--reboot-exit`)

15 Troubleshooting

Another eMerger run is in progress. A stale `flock` on `/tmp/emerger.lock`. Inspect `ps` for stragglers, then remove the lock file.

A manager fails with exit code 3. Run `up --errors` to tail the log, then `up --doctor` to inspect the environment. Most failures come from a manager's own health state (for example `dpkg` being interrupted); run the native repair command before retrying.

Nothing happens on scheduled runs. Check `systemctl --user status emerger.timer` or the Windows Task Scheduler entry. On Unix the scheduled run always uses `-y -q -nl -ni`; enable `-v` in the unit for one cycle to see the output.

16 Contributing

Pull requests and issues are welcome on GitHub. Before opening a PR, run the test suite in `tests/` on the platform you are touching, and make sure the feature is covered on both the Bash and the

PowerShell sides when it makes sense for both platforms.

If your question is not answered here, open an issue on GitHub: we would rather improve the documentation than answer the same question twice.